

RoK4 Reward Structure

작성일: 2026-07-15

이 문서는 RoK4-Isaac-Velocity-Flat-v0 task의 현재 reward 구조와 reward function 설정을 정리한다. 현재 reward는 Isaac Lab G1 velocity task 구조를 RoK4 링크/관절 이름에 맞게 이식한 flat walking 초기 baseline이다. 최종 튜닝값이 아니라 첫 보행 실험을 위한 시작점으로 봐야 한다.

아래 경로에서 RoK4: 는 /home/rcrlab/rok4_lab 을, Isaac Lab: 은 /home/rcrlab/IsaacLab 을 기준으로 한 상대경로를 뜻한다.

관련 파일

역할	파일
RoK4 flat task 설정	RoK4: .../config/rok4/flat_env_cfg.py
RoK4 ADAPT transmission/actuator	RoK4: .../assets/robots/rok4_adapt.py
RoK4 actuator action/observation	RoK4: .../velocity/mdp/actions.py, observations.py
RoK4 actuator reward 함수	RoK4: .../velocity/mdp/rewards.py
RoK4 domain randomization 설정	RoK4: .../config/rok4/domain_randomization_cfg.py
공통 locomotion reward term 기본값	Isaac Lab: .../velocity/velocity_env_cfg.py
locomotion 전용 reward 함수	Isaac Lab: .../velocity/mdp/rewards.py
Isaac Lab 공통 reward 함수	Isaac Lab: isaacclab/envs/mdp/rewards.py
mdp namespace 연결	Isaac Lab: .../velocity/mdp/__init__.py

Actuator interface와 reward 연결

현재 reward는 단순히 joint 값을 actuator라고 이름만 바꾼 것이 아니다. action target, 실제 상태, PD torque가 각각 ADAPT 행렬을 통과하며, reward는 그 runtime actuator 값 또는 mapped joint target을 명시적으로 사용한다.

```
rok4.py
├─ 링크 길이, actuator gain/action scale/limit 설정
│   └─> rok4_adapt.py
│       ├── RoK4AdaptTransmission: J/J~-1/J^T/J~-T
│       └─ RoK4AdaptActuator: actuator PD와 tau_psi/tau_q 계산
│           ├──> mdp/actions.py
│           │   raw action -> psi_target -> q_target
│           ├──> mdp/observations.py
│           │   q/qdot -> psi/psi_dot observation
│           └─> mdp/rewards.py
│               ├── tau_psi, psi_dot, psi_ddot penalty
│               ├── actuator torque/velocity limit 초과 penalty
│               ├── scaled actuator action-rate penalty
│               └─ q_target joint-position-limit penalty
│                   └─> flat_env_cfg.py의 RoK4RewardsCfg
│                       func + weight + params를 RewardTerm으로 구성
```

rok4_adapt.py, actions.py, observations.py, rewards.py 는 모두 rok4_lab 안의 로컬 파일이며, Isaac Lab 원본 source를 수정하지 않는다. Isaac Lab의 부모 class와 mdp 함수는 import/상속/re-export해서 사용한다.

구조 요약

RoK4FlatEnvCfg 는 환경 설정이고, RoK4RewardsCfg 는 reward term 묶음이다. Reward 구조는 두 가지 관계로 나누어 보면 될 것같린다.

첫 번째는 config class의 상속/포함 관계이다.

```
RoK4FlatEnvCfg
├─ 상속: LocomotionVelocityRoughEnvCfg
│   └─ 위치:
│       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
└─ rewards: RoK4RewardsCfg()
    └─ RoK4RewardsCfg
        ├── 상속: RewardsCfg
        │   └─ 위치:
        │       IsaacLab/.../locomotion/velocity/velocity_env_cfg.py
        └─ 부모 RewardsCfg에서 물려받은 reward terms
```

```

├── lin_vel_z_l2
├── ang_vel_xy_l2
├── flat_orientation_l2
├── undesired_contacts
└── RoK4RewardsCfg에서 새로 정의/override한 reward terms
    ├── termination_penalty
    ├── track_lin_vel_xy_exp
    ├── track_ang_vel_z_exp
    ├── feet_air_time
    ├── feet_slide
    ├── dof_pos_limits
    ├── joint_action_target_pos_limits
    ├── joint_deviation_hip
    ├── joint_deviation_torso
    ├── actuator_acc_l2
    ├── actuator_torques_l2
    ├── actuator_vel_l2
    ├── actuator_velocity_limits
    ├── actuator_torque_limits
    ├── action_rate_l2
    └── second_action_rate_l2

```

두 번째는 각 reward term이 실제 계산 함수를 참조하는 관계이다.

```

RoK4FlatEnvCfg
├── rewards = RoK4RewardsCfg()
│   └── RoK4RewardsCfg extends RewardsCfg
│       └── 각 reward term은 RewardTermCfg/RewTerm
│           └── func=mdp.xxx
│               ├── RoK4 로컬 mdp 함수
│               │   └── source/rok4_tasks/.../velocity/mdp/rewards.py
│               ├── Isaac Lab 공통 mdp 함수
│               │   └── IsaacLab/source/isaacsim/isaacsim/envs/mdp/rewards.py
│               └── locomotion velocity 전용 mdp 함수
│                   └── IsaacLab/source/isaacsim_tasks/.../velocity/mdp/rewards.py

```

즉 RoK4FlatEnvCfg -> RoK4RewardsCfg 는 상속이 아니라 포함/사용 관계이고, RoK4RewardsCfg -> RewardsCfg 는 상속 관계이다. mdp/rewards.py 는 부모 클래스가 아니라 실제 reward 계산 함수가 들어 있는 함수 모음이다. RoK4는 로컬 rok4_tasks...velocity.mdp 를 import하며, 이 로컬 mdp는 Isaac Lab의 기존 locomotion mdp를 다시 export하고 RoK4 전용 reward 함수만 추가한다.

실제로는 다음처럼 이해하면 된다.

```

RoK4RewardsCfg는 RewardsCfg를 상속받아서 기본 reward term들을 물려받는다.
RoK4에 필요한 reward는 mdp.xxx 함수를 직접 지정해 새로 정의하거나 override한다.
RewardTermCfg/RewTerm은 mdp.xxx 함수, weight, params를 하나의 reward term으로 묶는다.

```

파일 배치 기준은 다음과 같다.

```

flat_env_cfg.py
├── 어떤 reward term을 쓸지, weight를 얼마로 둘지, 어떤 body/joint에 적용할지 결정

velocity/mdp/rewards.py
├── reward raw value를 어떻게 계산할지 정의

```

따라서 joint_action_target_pos_limits, actuator_torques_l2, actuator_vel_l2, actuator_acc_l2, actuator_velocity_limits, actuator_torque_limits, action_rate_l2, second_action_rate_l2 처럼 RoK4 전용 weighting을 쓰는 계산식은 velocity/mdp/rewards.py 에 두고, weight를 얼마로 사용할지는 RoK4RewardsCfg 에 둔다. 이 term은 이전 action 두 개가 필요하므로 reset 직후 첫 두 policy step에서는 penalty를 0으로 둔다.

Action-rate reward의 action 기준

현재 RoK4Lab의 action_rate_l2 와 second_action_rate_l2 는 env.action_manager.action 을 기준으로 현재 action 과 history의 차분을 만든다. 이 action buffer 자체는 scale 적용 전 raw policy action이지만, reward 함수는 차분에 ROK4_ACTUATOR_ACTION_SCALE 을 곱한 뒤 제곱합을 계산한다.

다만 RoK4FlatPP0RunnerCfg 에서 clip_actions = 1.0 을 명시하므로 Isaac Lab train.py 와 play.py 를 통해 실행할 때는 Rs1R1VecEnvWrapper 가 action을 먼저 [-1, 1] 로 clamp한다. 따라서 reward가 보는 action은 wrapper에서 [-1, 1] 로 잘린 raw actuator action이며, penalty가 측정하는 값은 scaled actuator-target offset 차분이다.

```

actor output
-> clip_actions = 1.0
-> clipped_raw_actuator_action
├── last_action observation = clipped_raw_actuator_action
└── scaled_actuator_offset = clipped_raw_actuator_action * ROK4_ACTUATOR_ACTION_SCALE
    ├── action_rate_l2 / second_action_rate_l2
    └── psi_target = psi_default + scaled_actuator_offset

```

```
└─ q_target = J * psi_target
   └─ joint_action_target_pos_limits compares q_target with soft_joint_pos_limits
```

여기서 `q_target` 은 PhysX position drive에 직접 입력되는 값이 아니라 explicit actuator model의 입력이다. `RoK4AdaptActuator.compute()` 가 `q_target` 과 현재 joint state를 actuator 좌표로 변환해 `tau_psi` 를 계산·제한하고, `tau_q = J-T tau_psi` 로 바꾼 joint effort만 PhysX에 전달한다. Actuator torque는 최대값의 90%에서 제어 및 reward 기준으로 제한하고, PhysX `effort_limit_sim` 은 factor를 적용하지 않은 joint mechanical maximum을 최종 solver 안전 상한으로 사용한다.

기존 IsaacGym RoK4 코드에서는 `actuator_actions_raw * action_scale` 으로 만든 `actuator_actions` 를 `penalty_action_rate` 와 `penalty_action_2nd_rate` 에 넘겼다. 즉 Gym 버전은 scaled actuator action 기준이고, 현재 RoK4Lab도 동일하게 scaled actuator action 차분을 기준으로 한다. 또한 `ROK4_ACTUATOR_ACTION_SCALE` 값도 Gym의 `[0.4, 0.5, 1.25, 1.5, 0.75, 0.75]` 좌우 다리 scale과 torso 0.4 에 맞췄다. 다만 observation의 `last_action` 은 Isaac Lab convention에 따라 clipped raw action으로 유지한다.

이 변경은 tensor 크기만 보면 이전 joint-space policy와 같지만 action, joint/actuator position, velocity의 의미가 다르다. 기존 joint-space checkpoint를 resume하면 잘못된 좌표 의미로 학습되므로 actuator-interface checkpoint는 반드시 새 학습에서 생성한다.

Action target의 `default_joint_pos` 도 Gym의 활성 gait-ready pose와 동일하다. Hip pitch는 -0.0924 rad, knee pitch는 0.345 rad, ankle pitch는 -0.253 rad 이며 나머지 관절은 0.0 rad 이다. 이 joint pose를 `psi_default = J-1 q_default` 로 변환한 값이 actuator action 중심이며, `joint_deviation * reward`는 계속 원래 joint default pose를 기준으로 한다.

Reward와 Domain Randomization의 구분

`RoK4RewardsCfg` 와 부모 `RewardsCfg` 는 reward term을 정의한다. 반면 foot friction, restitution, base/upper/lower body mass, base/upper/lower COM, reset pose 같은 domain randomization 값은 `domain_randomization_cfg.py` 에서 관리한다.

두 설정은 학습 결과에 모두 영향을 주지만 역할은 다르다.

구분	파일	역할
Reward	<code>flat_env_cfg.py</code> 의 <code>RoK4RewardsCfg</code> 및 <code>self.rewards.xxx</code>	policy가 어떤 행동을 더 좋게 볼지 점수 함수를 정의
DR	<code>domain_randomization_cfg.py</code>	물성, 질량, COM, 초기상태 perturbation을 바꿔 다양한 상황에서 버티게 함

예를 들어 발 미끄러짐은 `feet_slide` reward가 penalty로 줄이고, foot-ground 마찰 계수는 `domain_randomization_cfg.py` 의 `ROK4_STATIC_FRICTION_RANGE` 와 `ROK4_DYNAMIC_FRICTION_RANGE` 가 정한다. 따라서 미끄러짐 문제를 볼 때는 reward 와 DR을 함께 확인해야 하지만, 코드상 관리 위치는 분리되어 있다.

Reward와 PPO entropy/KL의 구분

Reward term은 환경의 `RewardManager` 가 행동의 점수를 계산하는 항목이다. 반면 PPO의 entropy와 KL은 `rs_l_rl_optimizer`가 policy distribution을 업데이트할 때 사용하는 학습 지표다. 따라서 `entropy_coef` 나 `desired_kl` 은 `RoK4RewardsCfg` 의 reward weight가 아니다.

Reward와 PPO 지표 비교

항목	현재 값/위치	역할
reward weight	<code>flat_env_cfg.py</code> 의 <code>RewTerm(weight=...)</code>	환경 행동의 positive reward 또는 penalty 크기를 결정
entropy_coef	<code>rs_l_rl_ppo_cfg.py</code> 의 0.002	Gaussian policy의 exploration entropy를 유지하려는 optimizer 압력을 결정
desired_kl	<code>rs_l_rl_ppo_cfg.py</code> 의 0.01	old/new policy 차이를 기준으로 adaptive learning rate를 조절

RoK4 로컬 `scripts/rs_l_rl/rok4_ppo.py` 는 adaptive schedule에서 이미 계산되는 mini-batch KL을 모아 TensorBoard의 `Loss/kl` (iteration 평균)과 `Loss/kl_max` (iteration 최대)로 기록한다. 현재 5 learning epochs와 4 mini-batches를 사용하므로 iteration당 20개 KL 값을 집계한다. 이 로깅을 위해 Isaac Lab 또는 설치된 `rs_l_rl` 파일을 수정하지 않는다.

Reward 계산 방식

Isaac Lab의 `RewardManager` 는 각 reward term의 raw value를 계산한 뒤 weight 를 곱하고, 모든 term을 합산한다.

```
total_reward =
    sum(weight_i * reward_function_i(env, params_i))
```

따라서 positive weight는 행동을 유도하고, negative weight는 penalty로 작동한다.

RoK4 전용 Reward Terms

아래 term들은 `RoK4RewardsCfg` 에서 직접 정의하거나 부모 `RewardsCfg` 의 term을 RoK4용으로 override한 항목이다.

term	function	weight	params	의미
termination_penalty	mdp.is_terminated	-200.0	없음	episode가 terminated penalty를 준다
track_lin_vel_xy_exp	mdp.track_lin_vel_xy_yaw_frame_exp	1.0	command_name="base_velocity", std=0.5	yaw-aligned linear velocity x/y 선속도 covariance를 준다
track_ang_vel_z_exp	mdp.track_ang_vel_z_world_exp	1.0	command_name="base_velocity", std=0.5	world frame angular velocity covariance를 준다
feet_air_time	mdp.feet_air_time_positive_biped	0.75	feet: L_Foot_Link, R_Foot_Link; threshold=0.55	양발 보행에서 step pattern이 있는 step pattern의 Threshold는 0.55가 아니라 sin wave의 시간 상한이다
feet_slide	mdp.feet_slide	-0.2	feet: L_Foot_Link, R_Foot_Link	지면 접촉 중이 아닐 때 발생하는 것을 줄인다
dof_pos_limits	mdp.joint_pos_limits	-1.0	ROK4_JOINT_ORDER에 포함된 전체 13관절, preserve_order=True	각 관절이 US 95%로 계산된 joint position을 넘은 양을 합친다
joint_action_target_pos_limits	mdp.joint_action_target_pos_limits	-0.001	actuator_pos action의 mapped joint target과 ROK4_JOINT_ORDER의 전체 13관절	Gym의 joint position penalty action을 대응한다. ADA의 joint position을 넘은 양을 합친다
joint_deviation_hip	mdp.joint_deviation_l1	-0.1	.*_Hip_Yaw_Joint, .*_Hip_Roll_Joint	hip yaw/roll angle에서 과도하게 벗어난다
joint_deviation_torso	mdp.joint_deviation_l1	-1.0	Torso_Yaw_Joint	torso yaw가 과도하게 벗어난다
actuator_acc_l2	mdp.actuator_acc_l2	-1.0e-8	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	Isaac Lab의 joint acceleration을 J ⁻¹ 로 acceleration을 제한한다. rad/s ² 값은 undivided velocity에 따른다
actuator_torques_l2	mdp.actuator_torques_l2	-1.0e-5	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	ADAPT exploration torque-limit constraint tau_psi의 weighted sum이다
actuator_vel_l2	mdp.actuator_vel_l2	-1.0e-4	전체 RoK4 actuator; index 2,3,8,9 weight 0.5	psi_dot = J ⁻¹ weighted square Gym penalty에 대응한다
actuator_velocity_limits	mdp.actuator_velocity_limits	-0.001	velocity_limit_factor=0.9를 적용한 actuator velocity limits	abs(psi_dot)가 velocity limit를 초과한다
actuator_torque_limits	mdp.actuator_torque_limits	-1.0e-5	torque_limit_factor=0.9를 적용한 actuator torque limits	clip 전 요청 torque limit을 초과해 saturation을 발생시킨다
action_rate_l2	mdp.action_rate_l2	-0.1	action 전체; index 2,3,8,9 weight 0.5	clipped_raw_action ROK4_ACTUATOR의 1차 차분을 pitch/knee acceleration을 완화한다
second_action_rate_l2	mdp.second_action_rate_l2	-0.05	action 전체; index 2,3,8,9 weight 0.5	scaled action: s(a_t - 2 a_{t-2})를 완화한다. resampling policy step은 0으로 처리

Gym과 Lab의 action-limit 신호 차이

Gym과 현재 Lab 구현 모두 actuator action을 ADAPT 변환한 mapped joint target을 검사한다. 현재 Lab의 RoK4ActuatorPositionAction.processed_actions가 q_target = J⁻¹ psi_target을 보관하므로 joint_action_target_pos_limits는 실제 PD command에 대응하는 joint target을 검사한다.

Gym과 Lab의 torque/velocity 신호 차이

기존 Gym 코드와 현재 Lab 코드는 모두 ADAPT transmission 변환 전 actuator torque/velocity를 penalty에 사용한다. 현재 Lab custom actuator는 applied_actuator_effort 를 직접 노출하고 joint velocity를 J^{-1} 로 변환한다. 따라서 torque/velocity reward 좌표계와 0.5 weighting은 Gym과 동일하다. Acceleration만 현재 Lab에서 $J^{-1} \ddot{q}$ 의 실제 acceleration을 사용하므로 Gym의 $\psi_{dot,t} - \psi_{dot}(t-1)$ 과 정의가 다르다.

부모 RewardsCfg에서 상속받고 RoK4에서 수정한 Terms

아래 term들은 RewardsCfg 에서 기본으로 제공되며, RoK4FlatEnvCfg.__post_init__() 에서 weight나 body/joint 대상이 RoK4에 맞게 수정된다.

term	function	current weight	RoK4 설정	의미
lin_vel_z_l2	mdp.lin_vel_z_l2	-0.2	기본 robot root	base가 z 방향으로 튀는 움직임을 줄인다.
ang_vel_xy_l2	mdp.ang_vel_xy_l2	-0.05	기본 robot root	roll/pitch angular velocity를 줄인다.
flat_orientation_l2	mdp.flat_orientation_l2	-10.0	기본 robot root	몸이 기울어지는 것을 줄이고 upright 자세를 유도한다.
undesired_contacts	mdp.undesired_contacts	-1.0	Base_Link, Upper_Body_Link; threshold=1.0	발이 아닌 base/upper body 접촉을 penalty로 처리한다.

Reward Function 요약

function	위치	계산 의미
is_terminated	Isaac Lab 공통	termination manager의 terminated flag를 reward term으로 반환한다.
track_lin_vel_xy_yaw_frame_exp	locomotion mdp	yaw frame x/y 선속도 command error에 $\exp(-error / std^2)$ 를 적용한다.
track_ang_vel_z_world_exp	locomotion mdp	world z축 angular velocity command error에 $\exp(-error / std^2)$ 를 적용한다.
feet_air_time_positive_biped	locomotion mdp	한 발 지지 상태의 air/contact time을 보상하며, command 크기가 작으면 0이 된다.
feet_slide	locomotion mdp	접촉 중인 foot의 horizontal linear velocity norm을 penalty로 계산한다.
lin_vel_z_l2	Isaac Lab 공통	base body-frame z velocity squared.
ang_vel_xy_l2	Isaac Lab 공통	base body-frame roll/pitch angular velocity squared sum.
flat_orientation_l2	Isaac Lab 공통	projected gravity의 x/y component squared sum으로 몸 기울기를 penalty화한다.
actuator_torques_l2	RoK4 로컬 mdp	actuator-space applied torque weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_vel_l2	RoK4 로컬 mdp	$J^{-1} \dot{q}$ actuator velocity의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_acc_l2	RoK4 로컬 mdp	$J^{-1} \ddot{q}$ actuator acceleration의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
actuator_velocity_limits	RoK4 로컬 mdp	velocity_limit_factor 를 적용한 actuator velocity limit 초과량.
actuator_torque_limits	RoK4 로컬 mdp	clip 전 actuator torque command가 torque_limit_factor 적용 limit을 넘은 양.
action_rate_l2	RoK4 로컬 mdp	현재 action과 이전 action 차이의 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
second_action_rate_l2	RoK4 로컬 mdp	현재 action, 이전 action, 이전-이전 action의 2차 차분 weighted squared sum. Index 2,3,8,9 는 weight 0.5.
joint_deviation_l1	Isaac Lab 공통	현재 joint position과 default joint position 차이의 absolute sum.
joint_pos_limits	Isaac Lab 공통	soft joint position limit을 넘은 정도를 합산한다.
joint_action_target_pos_limits	RoK4 로컬 mdp	actuator_pos action이 ADAPT mapping한 joint target의 soft joint position limit 초과량.
undesired_contacts	Isaac Lab 공통	지정 body의 contact force가 threshold를 넘은 contact 개수를 penalty로 반환한다.

Command와 Reward의 연결

현재 RoK4 flat command range는 다음과 같다.

```
lin_vel_x = (-0.1, 0.85)
lin_vel_y = (-0.3, 0.3)
```

```
ang_vel_z = (-0.6, 0.6)
```

이 command는 track_lin_vel_xy_exp, track_ang_vel_z_exp, feet_air_time 에 직접 영향을 준다. 특히 feet_air_time_positive_biped 는 x/y command norm이 0.1 이하이면 reward를 0으로 만든다. 즉 거의 정지 명령에서는 stepping reward가 강하게 작동하지 않는다.

UniformVelocityCommand 가 반환하는 [lin_vel_x, lin_vel_y, ang_vel_z] 는 robot base frame 기준 command다. 따라서 양의 lin_vel_x 는 로봇이 현재 바라보는 방향의 전진, 양의 lin_vel_y 는 로봇 기준 좌측 이동, 양의 ang_vel_z 는 z축 주위 반시계 방향 회전을 뜻한다. 선속도 tracking 함수는 실제 world 선속도에서 roll/pitch를 제외한 yaw 회전만 역변환한 gravity-aligned yaw frame 속도와 x/y command를 비교한다. 반면 현재 각속도 tracking 함수는 command의 z 성분과 world-frame root_ang_vel_w[:, 2] 를 비교한다. 평평하고 직립한 상태에서는 두 z축이 거의 같지만, 몸체가 크게 기울면 완전히 동일한 표현은 아니다.

부모 command 설정의 heading_command=True 와 rel_heading_envs=1.0 도 그대로 적용된다. 따라서 학습 중 yaw command는 처음 표본화한 각속도를 그대로 유지하는 방식이 아니다. World frame target heading을 표본화하고, 매 step $0.5 * \text{wrap_to_pi}(\text{target_heading} - \text{current_heading})$ 을 계산한 뒤 현재 ang_vel_z 범위인 $(-0.6, 0.6)$ rad/s 로 clip한 결과를 base-frame command의 z 성분에 저장한다.

현재 feet_air_time 은 threshold=0.55 s, weight=0.75 를 사용한다. 따라서 최대 raw reward는 0.55 이고, 최대 pre-dt 항목 크기는 $0.55 * 0.75 = 0.4125$ 이다. 이 설정은 긴 single stance를 기존 threshold=0.4 보다 강하게 보상한다.

Contact sensor는 update_period=0.002 s 와 history_length=self.decimation=5 를 사용한다. 따라서 feet_slide, undesired_contacts, illegal_body_contact 처럼 net_forces_w_history 를 검사하는 항목은 최근 policy interval의 5 개 physics contact sample을 확인한다. 반면 feet_air_time_positive_biped 는 이 force history가 아니라 sensor의 현재 current_air_time 과 current_contact_time 을 사용한다. 이 contact-force history는 policy observation history와 별개의 buffer이다.

Play 환경은 lin_vel_x=0.85 m/s 를 사용하고 lateral/yaw command는 0으로 고정한다. 별도의 Teleop 환경은 heading_command=False 와 긴 resampling interval을 사용하고, gamepad 또는 keyboard의 [lin_vel_x, lin_vel_y, ang_vel_z] 를 동일한 base-frame command buffer에 직접 기록한다. 입력은 학습 범위 $(-0.1, 0.85)$ m/s, $(-0.3, 0.3)$ m/s, $(-0.6, 0.6)$ rad/s 안으로 scale된다. 이는 inference command source만 바꾸며 reward 정의나 학습 checkpoint 자체를 변경하지 않는다.

Termination과 Reward의 연결

RoK4 flat task는 Isaac Lab의 TerminationsCfg 를 수정하지 않고 로컬 RoK4TerminationsCfg 로 상속한다. 부모의 time_out 은 유지하고 base_contact 는 비활성화한 뒤 illegal_body_contact 를 추가한다.

```
class RoK4TerminationsCfg(TerminationsCfg):
    base_contact = None
    illegal_body_contact = DoneTerm(
        func=mdp.illegal_contact,
        params={
            "sensor_cfg": SceneEntityCfg(
                "contact_forces",
                body_names=ROK4_ILLEGAL_CONTACT_BODY_NAMES,
            ),
            "threshold": 1.0,
        },
    )
```

ROK4_ILLEGAL_CONTACT_BODY_NAMES 에는 Base_Link, Upper_Body_Link 와 좌우 Hip_Yaw, Hip_Roll, Thigh, Calf, Ankle_Pitch, Ankle_Roll link가 포함된다. 정상 접촉 body인 L_Foot_Link 와 R_Foot_Link 만 제외된다. 선택된 body 중 하나라도 contact force가 1.0 N 을 넘으면 episode가 종료된다.

mdp.is_terminated 는 non-timeout termination을 감지하므로 illegal_body_contact 가 발생하면 termination_penalty=-200.0 가 함께 작동한다. 반면 부모에서 상속한 정상 time_out 에는 이 penalty가 적용되지 않는다. 여러 body가 동시에 접촉해도 termination flag는 boolean이므로 termination penalty는 한 번만 적용된다. Contact sensor 는 상대 물체 종류를 구분하지 않은 net contact force를 사용하므로 self-collision으로 선택 body에 큰 접촉력이 생겨도 종료될 수 있다.

현재 설계 의도

현재 reward는 다음 목표를 동시에 가진다.

1. command velocity를 따라간다.
2. upright 자세를 유지한다.
3. 한 발씩 드는 biped stepping pattern을 만든다.
4. 발 미끄러짐을 줄인다.
5. torque, joint acceleration, action rate, second action rate를 줄여 움직임을 부드럽게 한다.
6. ankle limit, hip/torso deviation을 제한한다.
7. Foot를 제외한 body 접촉을 실패 종료로 처리한다.

튜닝 시 우선 확인할 항목

현상	먼저 볼 항목
너무 자주 넘어진다	termination_penalty, illegal_body_contact, self-collision, flat_orientation_l2, 초기 자세
거의 걷지 않고 버틴다	track_lin_vel_xy_exp, command range, feet_air_time
발이 많이 미끄러진다	feet_slide, foot collision, friction, contact sensor
관절이 떨린다	action_rate_l2, second_action_rate_l2, actuator_acc_l2, actuator PD gain, action scale
토크가 과도하다	actuator_torques_l2, actuator torque limit, action scale
관절이 limit 근처로 간다	dof_pos_limits, joint_action_target_pos_limits, 해당 joint의 default pose, action scale, USD hard limit